

Facilitating Task Breakdown in Sprint Planning Meeting 2 with an Interaction Room

An Experience Report

Simon Grapenthin

paluno – The Ruhr Institute for Software Technology
University of Duisburg-Essen
45127 Essen, Germany
simon.grapenthin@paluno.uni-due.de

Steven Poggel

adesso AG
Stockholmer Allee 24
44269 Dortmund, Germany
steven.poggel@adesso.de

Matthias Book

paluno – The Ruhr Institute for Software Technology
University of Duisburg-Essen
45127 Essen, Germany
matthias.book@paluno.uni-due.de

Volker Gruhn

paluno – The Ruhr Institute for Software Technology
University of Duisburg-Essen
45127 Essen, Germany
volker.gruhn@paluno.uni-due.de

Abstract— In a large industry project that followed an agile approach based on the Scrum method, we found that the team often struggled with breaking coarse product backlog items down into the detailed tasks that had to be completed in a sprint. The team's understanding of the backlog items' business and technical implications and dependencies seemed not deep enough to identify all necessary tasks, so in addition to the tasks defined in the sprint planning meeting prescribed by Scrum, an average of 26% of additional tasks was identified later over the course of each sprint, making any attempt at progress estimation or risk management very difficult. To counter this effect and support the team's understanding of backlog items right from the beginning of each sprint, we introduced a pragmatic method to analyze backlog items more comprehensively and thus support a more complete and reliable task breakdown. We consequently found that the effectiveness and precision of task breakdowns has improved significantly in the project.

Keywords— *Scrum, Task Breakdown, Interaction Room*

I. INTRODUCTION

Scrum is the most popular agile software process model [1]. It structures a project into a series of iterations that last from one to four weeks. Each iteration comprises the steps of iteration planning (in the *sprint planning meeting*), implementing and testing a certain product increment (in a *sprint*), presenting the results to stakeholders (in the *sprint review*) and reflecting on one's work in order to optimize future sprints (in the *sprint retrospective*) [2, 3]. Scrum defines three roles that together form the so-called Scrum team: The *product owner* is responsible for managing the product, defining and prioritizing product requirements [4, 5]. The *Scrum master* manages the process. He oder she protects the team from interferences, identifies and eliminates impediments and advocates Scrum within the organization [4, 5]. The *development team* consists of particular roles needed for the development (e.g. developer, tester and architect). The

development team (henceforth: "team") is a proper subset of the Scrum team, excluding the Scrum master and the product owner. The development team members manage themselves in terms of how they complete the work to be done in a sprint [4, 5]. The definition of this work occurs in the sprint planning meeting that takes place at the beginning of each sprint. It is divided into two separate meetings: In sprint planning meeting 1, the team gathers all information needed to estimate and commit to the implementation of a set of prioritized *backlog items*. A backlog item (or item for short) is a functional requirement defined by the product owner. In sprint planning meeting 2, the team is then supposed to define all *tasks* that need to be performed in order to design, implement and test the backlog items selected for the sprint. Tasks describe the team's planned activities that are necessary in order to fulfill the functional requirements. For each sprint, the team commits to a limited number of backlog items, based on the estimates of the items' complexity and the team's velocity (i.e. the number of work units that can be handled by the team in a sprint). The velocity is a measurement that becomes more precise the more iterations a team completes, because its members gain experience in estimating the system's complexity and their own performance [6].

Deriving tasks from backlog items is a complex task, however. The team needs to have detailed knowledge about a variety of business and technical aspects of the information system under construction, as well as an understanding of the process and component landscape it is being integrated into, and the data structures it should rely on. Often, slicing tasks out of a backlog item is not straightforward. For example, if the specification of a backlog item is written from a user's perspective, as is often the case, the team additionally has to make the transfer from the user perspective to the technical perspective. A team's inability to identify all tasks initially at the time of sprint planning meeting 2 therefore not only means that some tasks are simply recognized later in the course of the

sprint, but also that the initial tasks may not be properly understood, and that they certainly do not take into account any dependencies with late-identified tasks. Besides the challenge of creating a clean overall system design under these circumstances, it is also difficult to measure the progress within a sprint, since the number of completed tasks (the *burndown*) cannot be used as a reliable metric to ensure that the work planned for a sprint gets done in its allotted timebox [7].

In this paper, we report on an industry project conducted at a medium-sized software company, where a Scrum team felt that their task breakdown was not as effective as it should be because many tasks were only identified over the course of the sprint, and not (as desired) in the initial sprint planning meeting 2. We therefore introduced the team to the so-called Interaction Room (an approach described in detail in [8, 9]) to support a more in-depth item analysis and task breakdown during sprint planning meeting 2. In the following sections, we give an overview of the project situation we initially encountered (Sect. II), describe the new approach we introduced (Sect. III), and report on the improvements we observed after applying it in the project (Sect. IV).

II. PROJECT EXPERIENCE

The reported experiences stem from a team within a large-scale software project in which an inventory maintenance system for health insurances is being developed. The system contains over 3000 user interface views covering a large variety of insurance cases. A number of project teams are simultaneously working on different components of this inventory system. The team we observed is a Scrum team of five developers on the development team, one Scrum master and one product owner. This team is both developing new modules for the system, and implementing change requests for existing modules. Each sub-project spans about three months. At the time of our research, the team was working on a component for the automated receipt, remodeling, storage and dispatching of insurance data from external partners of the insurance company. The individual features of this new component were set forth in backlog items that served as a basis for the team's implementation work. From the backlog items, the team sliced individual tasks to plan each sprint.

In Scrum theory, this slicing should all take place in the sprint planning meeting 2. Nonetheless, Schwaber predicts that only 60% of a sprint's tasks will be defined in this meeting, and assumes that 40% of the tasks will become apparent later during the sprint [2]. In our industry project, even less – about 20-30% of the tasks – were added later during the sprint. However, we believe this is still an undesirably high number of late-identified tasks: If the team has a deep understanding of the backlog items, it should be able to decompose them into virtually all the tasks necessary to deliver the required functionality. Nevertheless, if the team is unable to identify all tasks early, it is probably lacking information about the backlog items. In consequence, the team would be committing to backlog items it has not fully understood.

In the project we observed, the team was indeed trying to carve useful tasks out of backlog items largely by intuitive “gut feeling” rather than by a concrete methodical approach. As a

consequence, many tasks were not identified in the sprint planning meeting 2, but only became obvious later in the course of the sprint. We suspect that many Scrum teams are struggling with this problem when identifying concrete tasks from backlog items, which prompted us to come up with a more structured – but still pragmatic and agile – method.

Before we introduced our approach, the team had performed the sprint planning meeting 2 in separate groups (which actually deviates from Scrum practice “by the book”): The six developers formed three groups with two developers each. Each pair took backlog items from the issue tracking software to identify tasks for them, and noted these tasks on cards. The tasks on the cards were not discussed further with the rest of the team, but just put on a bulletin board as a reminder. Thus, two thirds of the team members did not know any details about the tasks for completing a particular backlog item, as they had not worked on identifying them. Tasks were formulated as short notes such as “create database table for medical service”, so only the developers who created the task knew exactly how to create the database table (e.g. which attributes, primary key, relations to other tables etc. it should have). For the other developers, most tasks needed clarification before they could work on them. In a few extreme cases towards the end of the sprint, none of the team members could remember what a certain task meant and how it was necessary for the completion of a particular backlog item. This indicates a clear lack of thorough understanding of the backlog items.

Another critical aspect was the expertise of the team members. The team we observed had a good distribution of knowledge and expertise among the team members. However, there were certainly some areas where one team member had stronger knowledge and more experience than another (e.g. in user interface design). Since items were framed vertically [6, 7] (every item involved the data, logic and presentation layer) but the two developers who sliced a backlog item into tasks might not have expertise in all those layers, the task breakdown did not benefit from many developers' expertise, and the set of identified tasks remained incomplete.

The challenges that this incomplete task breakdown posed in practice can best be illustrated by looking at the burndown charts the team produced over a number of sprints. A burndown chart measures progress during a sprint by showing the remaining effort (in terms of story points or number of tasks) on the vertical axis, while the horizontal axis shows the time elapsed in the sprint [7]. Since the developers did not manage to identify all tasks at the beginning of the sprint, it became inevitable that additional tasks had to be added during the sprint to really complete the backlog items. That rendered the burndown chart less meaningful and therefore less valuable. This effect can be seen in Fig. 1 where the total number of known tasks *increases* over the course of the sprint to exceed the number of initially identified tasks, instead of remaining constant. (Note that this does not constitute a violation of the requirements freeze that Scrum mandates during a sprint: The additional tasks do not represent commitments to new requirements (i.e. backlog items), but represent previously unrecognized work that turned out necessary in order to complete the backlog items in the sprint backlog.)

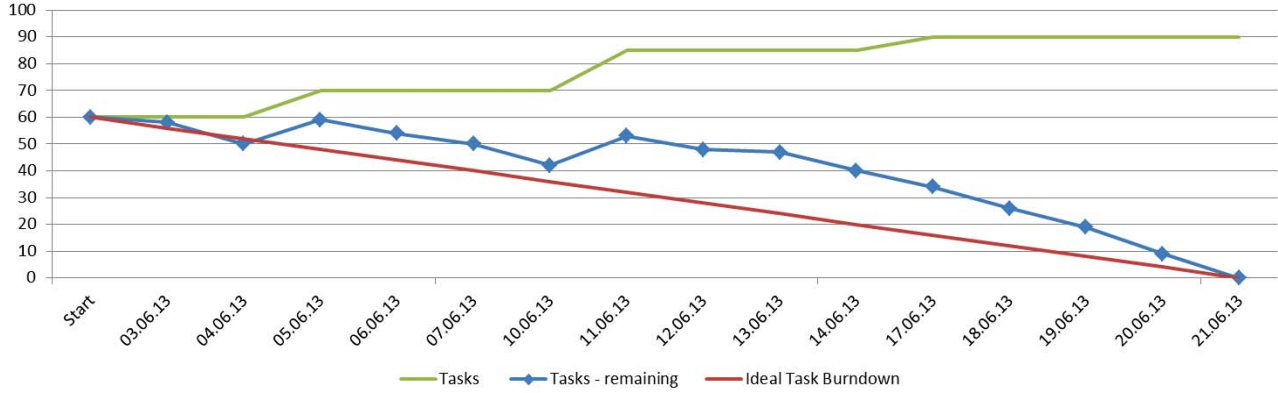


Fig. 1. Task burndown chart showing new tasks being identified over the course of the sprint.

If too many tasks are added subsequently during the sprint, it becomes much harder for the Scrum master (who is responsible for identifying and eliminating impediments to the team's work) to track progress based on the chart and to recognize and deal with projects risks, as Fig. 2 illustrates.

In this figure, the burndown charts of two sprints depict the situation before the introduction of our method. On the left of Fig. 2 is a burndown chart where 41 tasks were identified in sprint planning meeting 2, but another 16 tasks were identified over the course of the sprint, so 28% of tasks were added during the sprint. In the sprint charted on the right of Fig. 2, 38 tasks were initially identified in sprint planning meeting 2, but 12 more tasks were identified over the course of the sprint, so 24% of the tasks were added late. The total number of tasks increased steadily here as the team gained a deeper understanding of the backlog items and resulting tasks every day. On average, 26% of the tasks in this project's sprints were identified late in the sprints, instead of in sprint planning meeting 2 already.

These observations lead us to the conclusion that the team had difficulty with the derivation of tasks from backlog items. Thus, we looked for a method to support the team members' understanding of the backlog items without introducing a heavyweight design step into the agile process model.

III. SPRINT PLANNING 2 IN THE INTERACTION ROOM

In order to provide the team with a more structured, yet pragmatic approach to derive tasks from backlog items, we suggested that the complete team should work together on each item instead of forming pairs. Further, we encouraged the team to examine backlog items explicitly from four different perspectives that focus on static and dynamic, user- and system-focused aspects of each component and thus help the team to gain a comprehensive understanding of the system:

Firstly, since the main purpose of information systems is to support business processes, a backlog item should be described from a *business process* perspective. Secondly, as an information system is "an integrated set of components for collecting, storing, and processing data" [10], the second important perspective is the *data* perspective. Thirdly, an

information system is usually highly integrated into an application landscape and the system communicates with other systems through interfaces. Thus, the third perspective is an *integration* perspective. Besides, an information system is also a socio-technical system, so the team needs to consider backlog items from an *interaction* perspective. For each of these perspectives, we encouraged the team to examine backlog items and model the core process, data, integration and interaction aspects they entail. To facilitate this pragmatic, multi-perspective design sketching, we employed a so-called Interaction Room whose layout reflects the four perspectives [8, 9]. This can be a dedicated conference room or (as in our case) a section of the team room equipped with large whiteboards. Every whiteboard is dedicated to the representation of one perspective, as shown in Fig. 3:

- The *backlog* is adopted from agile practices [3]. It is used to manage the project's requirements (backlog items).
- The *process map* describes the sequence of business activities to execute the functionality described by a backlog item. It shows alternative and concurrent activity flows and relevant constraints.

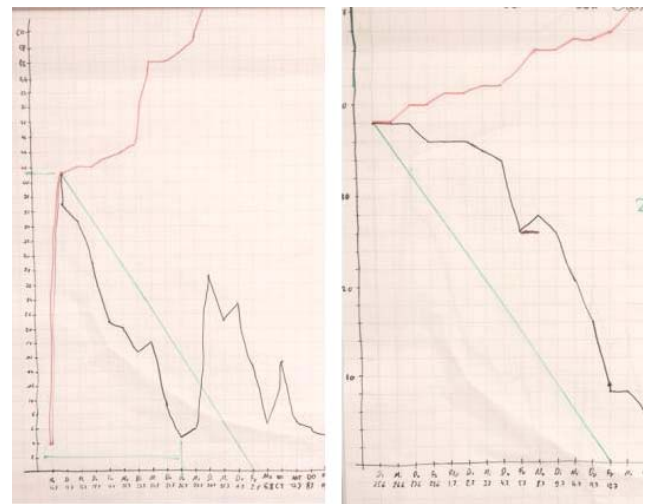


Fig. 2. Task burndown charts before the Interaction Room was introduced.

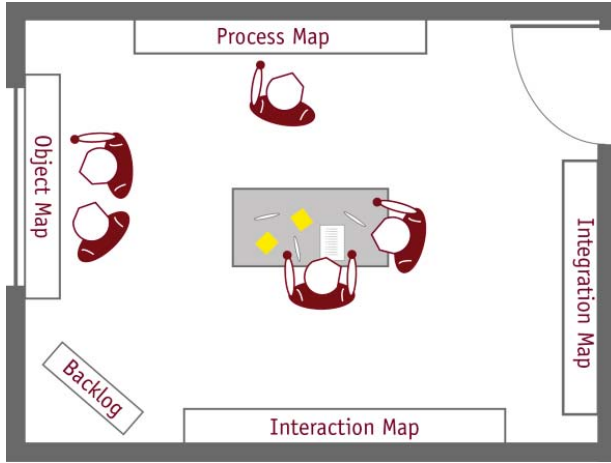


Fig. 3. Basic layout of an Interaction Room.

- The *object map* describes the internal data structure of the system, showing data objects on a high level of abstraction. It also shows relationships between data entities. The object landscape remains on a business level, so implementation details, such as composition dependencies, are left out.
- The *integration map* shows the interfaces of the system under development to other systems. Incoming and outgoing interfaces show the exchanged data and possibly the communication protocols.
- The *interaction map* describes the user interaction on different levels of abstraction. A navigation overview highlights the menu structure and underlying dialog

flows. Additionally, the graphical user interface can be sketched as a storyboard.

The model sketches in these maps remain deliberately abstract, informal and incomplete, as they are not intended to replace formal system specifications. Rather, they should serve as a catalyst for the discussion and mutual understanding of a project and its possible pitfalls. We deliberately talk about “maps” instead of “models” to emphasize their main goal: empowering users to understand and discuss the crucial parts of a system, and use the model sketches as maps to orient themselves in the project. For this purpose, we also do not prescribe particular modeling languages for the various maps, but let teams choose a notation that all stakeholders understand. This is in accordance with Cohn, who states that there should be some kind of design in sprint planning meeting 2, but recalls that he has “never been in an iteration planning meeting where it’s become necessary to draw [a] class diagram or similar model” [7].

The sprint planning meeting 2 was timeboxed between one and four hours (depending on the sprint length). Thus, the team had limited time to make an item design explicit and to identify the tasks to implement and test it. We therefore discouraged team members from attempting to create an explicit, complete technical design model for each backlog item in this short timeframe, but rather recommended that they focus on understanding the subject matter through model sketches and getting a more solid impression of the technical steps required to implement a particular item. Fig. 4 depicts the coarse methodical guideline for examining backlog items and deriving tasks from them.

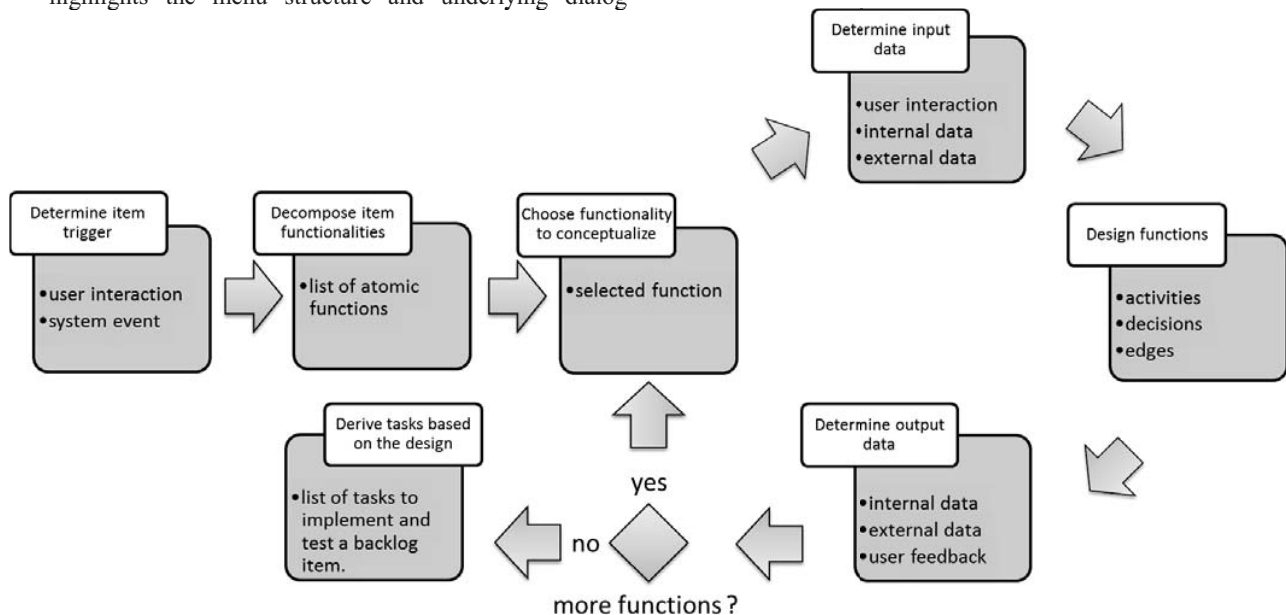


Fig. 4. Guideline for designing backlog items from different perspectives in order to identify tasks.

To discuss these perspectives in sprint planning meeting 2, we let the whole Scrum team gather together, not just (as previously done) the developers. This way, the product owner could work with the development team and provide feedback and information about the backlog items as required. The Scrum master moderated the meeting and encouraged the team members to participate in the discussion about the backlog items and the tasks (the Scrum master was chosen to moderate the meeting so the product owner would be on a level field with the developers). The developers discussed each backlog item based on its content: The team first decided if a backlog item would be triggered by a user or a system event. This decision was made by discussing the item between the development team members. For a user-triggered item, the team modeled the corresponding user interface views and sketched prototypes for them. For system-triggered items, the team modeled the event that triggers the business process. The Scrum master as the moderator was in charge of documenting the model sketches for later reference, but any team member was encouraged to actively join the design process.

Afterwards, the team decomposed the functionality of each item into its atomic functions. Once the atomic functions were identified, the team sketched a design for them by performing the following steps iteratively until the whole item was designed:

- Determine and model input data of the function
- Design function from a business process perspective
- Determine and model output data of the function

In case the input data would be provided by the user, the team modeled the user interaction and designed the data structures to store the provided data. On the other hand, if the input data would be provided by an outside system or component, the team designed the respective component interface. Since the output data could be stored as part of the internal data warehouse or written to a component or user interface, the team also sketched the respective data structures or interfaces in the Interaction Room.

The team repeated these steps of sketching designs from the data, process, integration and interaction perspective – not until they arrived at a complete specification, which would contradict the agile approach, but until all team members had gathered the necessary knowledge from the coarse sketches to understand the required functionality and its dependencies across the perspectives. With this knowledge, they could determine the tasks that would have to be performed in order to implement and test a backlog item. That way, the team should be much more likely to identify a more complete set of tasks for each item.

We believe that the task breakdown process cannot be described more formally because it is highly dependent on the nature of each backlog item. However, making the design of an item explicit by collaborating among all team members helps raising issues that could be overlooked if the design was made in small groups, or if there was no explicit consideration of the various design aspects in the sprint planning meeting at all.

By encouraging the Scrum team to examine the backlog items explicitly from the four perspectives described above, and visualizing the “big picture” relationships between multiple backlog items across these perspectives, we believe we can enable a Scrum team to perform their sprint planning meeting 2 in a more structured way. Instead of just extracting some tasks from backlog items with little comprehension of a backlog item’s details, they can gather information about all backlog items from different perspectives. That way, they gain a better understanding of each item’s technical implications. Based on the deeper insight, the team can identify tasks based on more solid ground of how to design, implement and test an item in the larger context of the overall system. In the following sections, we provide a short example to illustrate how the team worked with the Interaction Room, and report on the overall results of applying this approach in the project.

IV. INTERACTION ROOM USAGE EXAMPLE

Figure 5 shows an example of a process map and object map that were sketched by the team. The process map describes an automated process for sending a certain type of notification to customers of public health insurance companies.

The level of abstraction in this top-level sketch was quite high, as indicated by labels such as “searching for data”, “mapping of attributes to output structure”, “creation of output files”, “change of status” etc. After sketching the process map, the team went on sketching the corresponding object map. They went through the individual process steps and determined which data needed to be processed in each particular step. In sketching the object map, the team focused on the business objects that are processed in the software module, e.g. “bank account”, “application”, “insured person” etc. The team paid special attention to designing the maps in such a way that each entity shown on the object map was manipulated by some activity on the process map, and each entity known to be manipulated in the course of an activity on the process map was also noted on the object map.

With the understanding and knowledge gained from this modeling process, the team was convinced they knew enough about the module to be built in order to understand and estimate some of the backlog items. The team then went into greater detail and modeled the missing process steps on the process map, added missing objects to the object map and designed the user interface on the interaction map (not shown here for brevity). The integration map reflecting the technical implications of the process in this case comprised only one call to a particular service in a different software component.

After this, creating the individual tasks that were necessary to fulfill the acceptance criteria of a backlog item was astonishingly easy, compared to how the team had difficulty defining all the right tasks earlier. Because the whole process, its data structures, user interfaces and integration implications were now understood very well, the necessary tasks were clear to everyone, and just had to be documented on cards for the Scrum board.

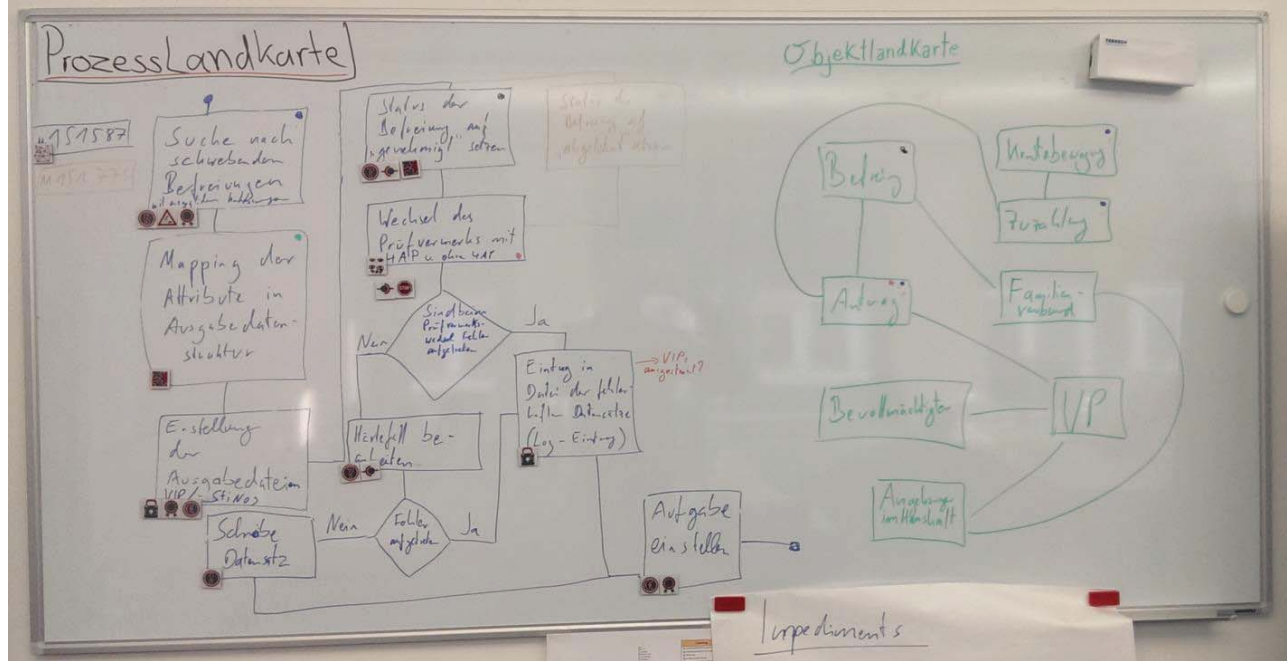


Fig. 5. Example of process and object map in the Interaction Room.

V. IMPACT IN PRACTICE

After introducing the Interaction Room to the Scrum team, we observed their performance over the course of three sprints to assess the impact of our approach. As an indicator for its benefits, we tracked the tasks that were added late during each sprint. In the three sprints, 19 backlog items were implemented. These items were broken down into 155 tasks, of which 145 tasks were derived in sprint planning 2 meetings and 10 tasks were added subsequently. Thus, in total only 6.8% of the tasks were added late during a sprint. Fig. 6 shows the burndown charts of the three observed sprints.

As can be seen in these charts, the top lines (i.e. the number of identified tasks) remain relatively level for most of the sprint duration, and rise only slightly on occasion (in contrast to the strongly rising top lines in the charts in Fig. 2). In the first sprint, the team committed to implement six backlog items. In sprint planning meeting 2, 48 tasks were identified for these, and seven tasks were added during the sprint, i.e. 12% of tasks were added subsequently. In the second iteration, seven backlog items were implemented, for which 53 tasks were derived during sprint planning meeting 2. Just one task was added later during the sprint, i.e. 2% of the tasks were added subsequently. In the third iteration, six backlog items were committed to, for which 44 tasks were derived during sprint planning meeting 2. Two additional tasks were added during the sprint, i.e. 5% of the tasks were added late here (Table I).

Our observations of the sprints before and after the introduction of the Interaction Room show that the percentage of late-identified tasks fell from the previous average of 26% to less than 6.8% on average after the new approach had been introduced – much less than the 40% late-identified tasks that

even Schwaber considered inevitable [2]. We hypothesize that this is due to the more comprehensive examination of each backlog item from the four perspectives of the Interaction Room, and in particular the discussion sparked among the team members while considering and modeling a backlog item's structural and functional implications in the overall system.

TABLE I. TASK BREAKDOWN RESULTS IN THE INTERACTION ROOM

Sprint	total number of identified tasks	late-identified tasks	percentage of late-identified tasks
1	48	7	12%
2	53	1	2%
3	44	2	5%

From the experience with three sprints in this project alone, we obviously cannot claim with certainty that the observed improvement in task breakdown efficiency was caused solely by the introduction of the Interaction Room – two other potentially contributing factors that one could think of would be the team members' increasing familiarity with the project matter, as well as varying complexity of the sprints' tasks. However, the general level of task complexity was quite homogeneously high across the various sprints, and the sharp drop to just about a quarter of the previous number of late-identified tasks seems inconsistent with a gradual familiarization process. We can also rule out an "observer effect" (i.e. the team trying particularly hard to identify all relevant tasks because if they know the task number to be a performance metric), since the team members were not aware that the number of late-identified tasks would be evaluated by us after introduction of the new method.

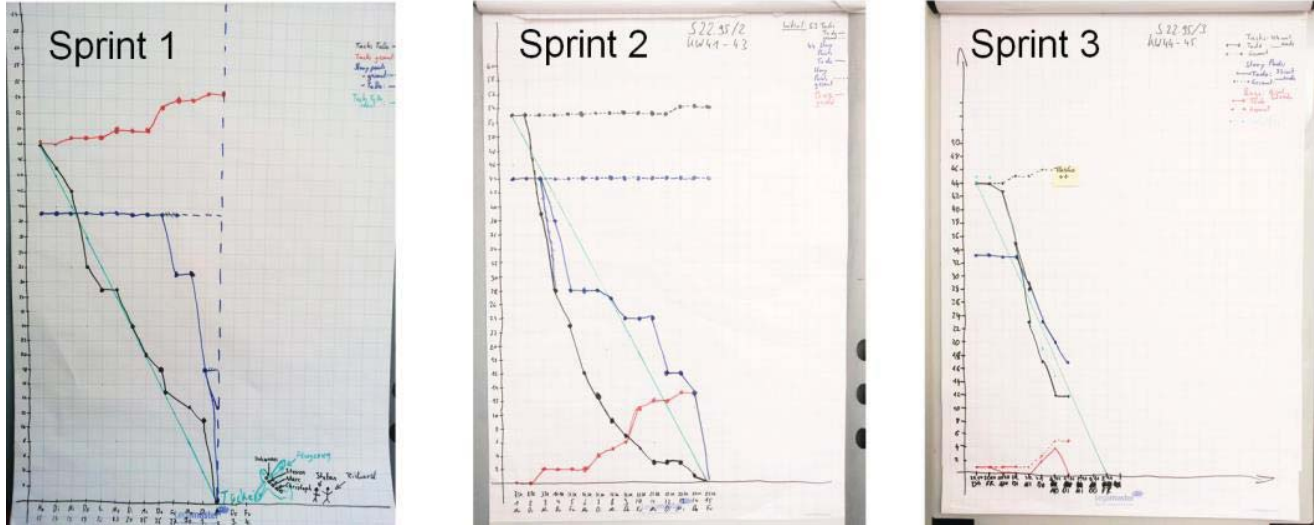


Fig. 6. Task burndown charts of the sprints after the Interaction Room was introduced.

Two additional side benefits of the approach are noteworthy: Firstly, since the team identified almost all tasks already in sprint planning meeting 2, it was possible to measure the sprint velocity much more reliably, since the number of tasks in the burndown chart remained largely stable, so the team members had a more realistic impression of the progress they actually made. Secondly, when using the Interaction Room, the whole team collaborated on the design sketches and the task decomposition, so every team member became able to understand or (with less extra study) even perform every task, whereas the relevant knowledge about individual tasks had been much more compartmentalized before.

In addition to the quantitative results, we also asked the team members to give us their informal opinion about the new approach. They responded that previously, they had sometimes felt uncertain why and how to perform a task during a sprint, but now felt that the new collaborative design fostered joint understanding about the backlog items, derived tasks and their rationales. This was particularly important in this project, since every backlog item was designed separately instead of in context of the whole system (and thus other backlog items). We believe that we could achieve further improvements in the completeness of the identified tasks if the relationships between various backlog items were taken into account, as these can be quite easily visualized in the Interaction Room.

VI. CONCLUSION AND OUTLOOK

In summary, our observation of a large Scrum-based industry project showed that about a quarter of all tasks was originally not identified in sprint planning meeting 2, but only became apparent later during the course of the sprint, indicating that the team members were having difficulty deriving all necessary tasks from a sprint's backlog items without any methodical support. This meant that the initially identified tasks were probably not well thought through, and that measuring sprint progress could not be accurate as the number of sprints per task was not stable. Also, the team faced

considerably more obstacles in the execution of the sprint due to the insufficient sprint planning.

We therefore encouraged team members to analyze backlog items and their resulting tasks from four different perspectives (business processes, data structures, integration and interaction) in an Interaction Room that facilitated pragmatic sketching and discussion of designs to obtain a high-level overview of the system. Using this approach, the whole team could arrive at a more comprehensive overview of the system, and identify the tasks necessary to implement and test individual backlog items significantly more confidently and completely – less than 7% of tasks were now added late during a sprint on average.

In our ongoing research, we are continuing to observe further sprints in this project to obtain further data on the impact of our approach on task breakdown. We will also adopt the Interaction Room in other Scrum-based projects as well, in order to obtain more data that will help us to generalize our findings and identify the methodical elements that are primarily responsible for the observed improvements. Furthermore, we will also investigate if there could be a complexity measure for backlog items that suggests when a more in-depth analysis in the Interaction Room is advisable.

- [1] Schwaber, C., Leganza, G., D'Silva, D.: The Truth About Agile Processes. Forrester Research (2007)
- [2] Schwaber, K.: Agile project management with Scrum. Microsoft Press, Redmond, WA (2004)
- [3] Schwaber, K., Beedle, M.: Agile Software Development with Scrum, 1st edn. Prentice Hall PTR, Upper Saddle River, NJ, USA (2002)
- [4] Rubin, K.S.: Essential Scrum. A practical guide to the most popular agile process. The Addison-Wesley signature series. Addison-Wesley, Upper Saddle River, NJ (2013)
- [5] Cohn, M.: Succeeding with Agile: Software Development Using Scrum. Pearson Education (2009)
- [6] Cohn, M.: User Stories Applied: For Agile Software Development. Addison Wesley (2004)
- [7] Cohn, M.: Agile Estimating and Planning. Prentice Hall PTR, Upper Saddle River, NJ, USA (2006)

- [8] Grapenthin, S., Book, M., Gruhn, V., Schneider, C., Völker, K.: Reducing Complexity Using an Interaction Room: An Experience Report. In: Proceedings of the 31st ACM International Conference on Design of Communication, pp. 71–76. ACM, New York, NY, USA (2013)
- [9] Book, M., Grapenthin, S., Gruhn, V.: Seeing the forest and the trees: focusing team interaction on value and effort drivers. In: Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, pp. 30:1-30:4. ACM, New York, NY, USA (2012).
- [10] Encyclopædia Britannica: Definition: information system (2013)